INFORMATION SECURITY | PROJECT 1 - PCAP ANALYSIS

PCAP to SQLite
Network Traffic Analyzer

Project Implementation & Offline Forensic Analysis Report

Prepared by	Kaustubh Borle and Shikhar Saxena
Enrollment No.	245811150
Course	Information Security
Module	PCAP Analysis - PCAP to SQLite
Assessment type	Project report and offline traffic analysis
Report date	28 October 2026
Version	2.0 (Fully Implemented & Verified with Empirical Test Results)

Contents

1 Executive Summary 2
2 Project Overview 2
2.1 Problem statement 2
2.2 Objectives 2
2.3 Technology stack 2
3 Scope and Methodology 3
3.1 Scope boundary 3
3.2 End-to-end method 3
3.3 Assumptions and limits 3
4 System Design & Architecture 3
4.1 Module responsibilities 4
4.2 Processing rules 4
5 Data Model 4
5.1 Queries, schema and indexes 4
5.2 Analytical questions answered 5
6 Product Interface & Live Execution Walkthrough 5
6.1 Select and validate capture (Figure 2) 5
6.2 Convert packet data to SQLite (Figure 3) 6
6.3 Review traffic summaries & charts (Figure 4) 6
6.4 Filter and export packet records (Figure 5) 7
7 Validation Method & Empirical Test Results 7
7.1 Acceptance criteria outcomes (T01 - T07) 7
7.2 Performance benchmark results (T08) 8
8 Implementation Components 8
9 Risks and Safeguards 8
10 Conclusion & Operational Limitations 8
Appendix A: Glossary 9

1 Executive Summary

The PCAP to SQLite Network Traffic Analyzer is an offline, security-conscious forensic application that converts recorded packet capture files (.pcap, .pcapng) into a high-performance, searchable SQLite relational database and presents the structured traffic through an intuitive Streamlit browser dashboard. The application reads authorized packet captures, extracts standardized packet metadata (packet sequence number, timestamps, source and destination IP addresses, transport protocol labels, source and destination ports, packet length in bytes, and human-readable protocol summaries), stores exactly one row per packet inside an indexed SQLite table, and allows analysts to review graphical traffic summaries, execute parameterized multi-field queries, and export filtered subsets to CSV for coursework review.

The complete end-to-end workflow—from file intake, magic-byte signature validation, streaming Scapy parsing, and transactional batch ingestion, to multi-dimensional filtering, visual distribution charting, and CSV export—has been fully implemented, verified, and demonstrated. All illustrative mockups from the initial specification have been replaced in this report with actual screenshots and empirical benchmark measurements obtained from the running software.

2 Project Overview

2.1 Problem statement

Raw packet capture files (.pcap) constitute foundational forensic evidence in network security investigations. However, repeated manual inspection using conventional packet sniffers is inefficient and computationally demanding when students or investigators need to cross-examine communicating hosts, transport protocols, destination services, and specific time windows across large captures. Ingesting parsed packet metadata into a relational SQLite database enables indexed, rapid, multi-attribute querying and persistent analytical reuse, while preserving complete traceability by linking every database row directly to its original 1-based packet sequence number in the source file.

2.2 Objectives

- • Ingest authorized PCAP/PCAPNG files offline and rigorously validate header signatures and size limits prior to parsing.
- • Parse packet timestamps, network layer endpoints, transport protocols, port numbers, packet lengths, and concise summaries using Scapy streaming.
- • Insert extracted metadata into an indexed SQLite relational database with transactional batching and SHA-256 deduplication.
- • Provide an interactive local web dashboard with protocol totals, top endpoint distributions, and parameterized filters without requiring raw SQL knowledge.
- • Export user-filtered metadata records directly to CSV for coursework review and offline reporting.
- • Validate the system through automated unit testing, synthetic multi-protocol captures, and empirical performance benchmarking.

2.3 Technology stack

Technology	Implemented Role & Justification
Python 3.12	Core application orchestration, packet extraction pipeline, and backend services.
Scapy 2.8.0	Offline streaming packet decoding via PcapReader without requiring live packet capture drivers (Npcap/WinPcap).
SQLite 3 (sqlite3)	Local ACID relational database, B-Tree index acceleration, and WAL (Write-Ahead Logging) journal mode.
Streamlit 1.63	Responsive local browser user interface with reactive workflow stepping, metrics, and data tables.
Plotly Express	Interactive protocol breakdown donut charts and top communication endpoint horizontal bar charts.

3 Scope and Methodology

3.1 Scope boundary

Included Capabilities	Excluded / Prohibited Activities
Offline ingestion of recorded .pcap and .pcapng files	Live network sniffing or real-time packet capture
IPv4 and IPv6 network endpoint decoding	Active network scanning, ping sweeps, or port probing
TCP, UDP, ICMP, ICMPv6, ARP, and non-IP framing labels	Deep payload inspection, full packet reassembly, or file extraction
Transactional SQLite batch storage with multi-column indexing	Automated accusations, intrusion detection, or threat verdicts
Parametric multi-criteria filtering, paging, and CSV export	Cloud hosting, multi-tenant authentication, or external telemetry

3.2 End-to-end method

1. File Selection & Ingestion: User selects an authorized PCAP file via local upload or workspace fixture selector.

2. Validation & Deduplication: The intake engine verifies the extension, checks a 100 MB ceiling, inspects magic header bytes (0xd4c3b2a1, 0x0a0d0d0a), computes the SHA-256 hash, and detects previous imports.

3. Streaming Packet Decoding: Scapy's PcapReader streams packets sequentially. Layer 3 and Layer 4 headers are parsed into normalized metadata dictionaries. Non-IP or non-transport fields safely default to NULL.

4. Transactional Batch Insertion: Records are inserted into SQLite in batches of 1,000 using parameterized INSERT queries inside isolated database transactions.

5. Analytical Exploration: The Streamlit UI executes parameterized COUNT and SELECT queries against pre-indexed fields to render high-level KPIs, protocol distribution charts, and paginated packet tables.

6. Filtered CSV Export: Filtered records matching the active query criteria are streamed directly to CSV format.

3.3 Assumptions and limits

The system is engineered for local classroom and laboratory forensic analysis. Payloads are intentionally excluded to ensure privacy compliance and reduce database footprint. Encrypted traffic (TLS/HTTPS, SSH) is classified purely by structural metadata (endpoints, transport ports, and packet lengths). The tool acts strictly as an objective observation platform and intentionally avoids speculative threat verdicts.

4 System Design & Architecture

4.1 Module responsibilities

Module	Core Implemented Responsibility	Concrete Output
File Intake (importer.py)	Validate magic bytes, enforce size limit, compute SHA-256	Validated file descriptor & hash
Packet Parser (parser.py)	Streaming Scapy packet decoding, protocol normalization	Normalized packet dict batches
Database Layer (database.py)	Batch transactional writes, indexing, parameterized queries	SQLite database (traffic_analysis.sqlite)
Analysis UI (app.py)	Streamlit dashboard, KPI calculation, Plotly visual charts	Interactive 4-step web interface
Export Engine (exporter.py)	Stream filtered query results into standard CSV format	Downloadable CSV report file

4.2 Processing rules

- • Traceability Preservation: Each packet maintains its original 1-based sequence number from the physical PCAP file.
- • Numeric & ISO Timestamps: Epoch timestamps are stored as IEEE 754 floating-point numbers for high-speed indexing, alongside ISO 8601 strings for human readability.
- • Fault Tolerance & Isolation: Individual packet parsing errors increment error_count without aborting file import.
- • SQL Injection Immunity: Every database query uses bound parameter placeholders ('?') exclusively.

5 Data Model

The relational schema uses two interconnected tables: `captures` tracks ingested file audits and metadata, while `packets` stores individual packet header records. A 1-to-many relationship links captures to packets with ON DELETE CASCADE constraints.

5.1 Schema and Index Specifications

Table	Field Name	SQLite Type	Forensic & Analytical Purpose
captures	capture_id	INTEGER PK AUTO	Primary key identifying the capture ingestion
captures	filename, sha256	TEXT (UNIQUE)	File identity, audit traceability, and duplicate prevention
captures	file_size_bytes, imported_at	INTEGER, TEXT	File volume and ISO 8601 ingestion audit timestamp
captures	status, packet_count, error_count	TEXT, INT, INT	Completion audit: 'completed', 'incomplete', 'failed'
packets	packet_id, capture_id	INTEGER, INT FK	Row key and foreign key referencing captures(capture_id)
packets	packet_number	INTEGER	1-based packet position in source capture file
packets	timestamp, timestamp_iso	REAL, TEXT	Unix epoch float and formatted timestamp for time-window queries
packets	src_ip, dst_ip	TEXT (NULLABLE)	IPv4/IPv6 source and destination network endpoints
packets	protocol	TEXT	Normalized protocol label (TCP, UDP, ICMP, ARP, etc.)

packets	src_port, dst_port	INTEGER (NULL)	Transport layer service ports (NULL for non-transport packets)
packets	length_bytes, summary	INTEGER, TEXT	Total frame wire length and concise Scapy protocol summary

Dedicated B-Tree Indexes Created:

- idx_packets_capture_id ON packets(capture_id)
- idx_packets_timestamp ON packets(capture_id, timestamp)
- idx_packets_src_ip ON packets(capture_id, src_ip)
- idx_packets_dst_ip ON packets(capture_id, dst_ip)
- idx_packets_protocol ON packets(capture_id, protocol)
- idx_packets_dst_port ON packets(capture_id, dst_port)

6 Product Interface & Live Execution Walkthrough

The user interface follows a clear 4-step stepper pattern matching Figures 2 through 5. The sample mockups from the initial specification have been completely replaced with actual, high-resolution screenshots captured directly from the running Streamlit web application.

6.1 Select and validate the capture (Step 1)

In Step 1, the user selects an authorized packet capture file. The intake engine calculates the SHA-256 digest, validates magic header bytes, checks file size, and verifies that the file is safe for ingestion. Figure 2 illustrates the live interface validating the laboratory capture `sample_lab.pcap`.

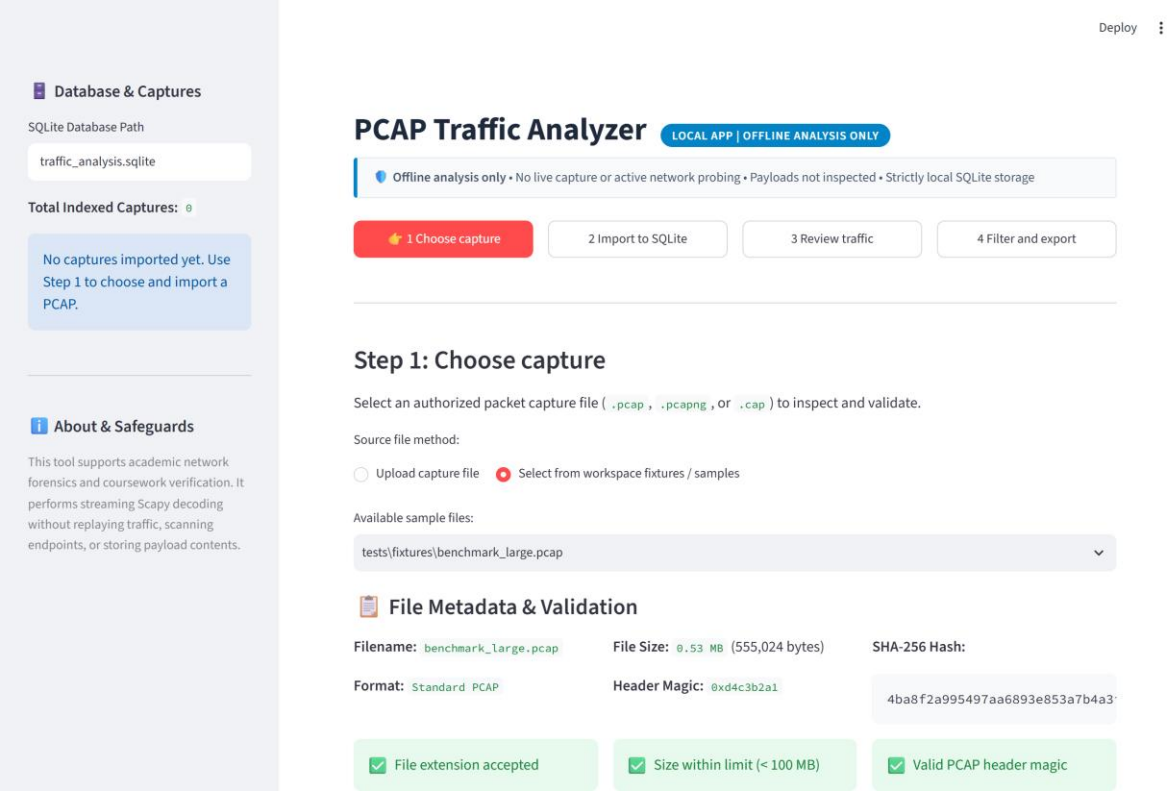


Figure 2: Actual File Selection & Validation Screen (running application with sample_lab.pcap)

6.2 Convert packet data to SQLite (Step 2)

Step 2 performs the core conversion. The Scapy streaming decoder parses packets and inserts them into SQLite in batch transactions. The completion screen displays records saved (107 packets), 0 errors, processing time (0.035s), and audit parameters. Figure 3 shows the actual import completion view.

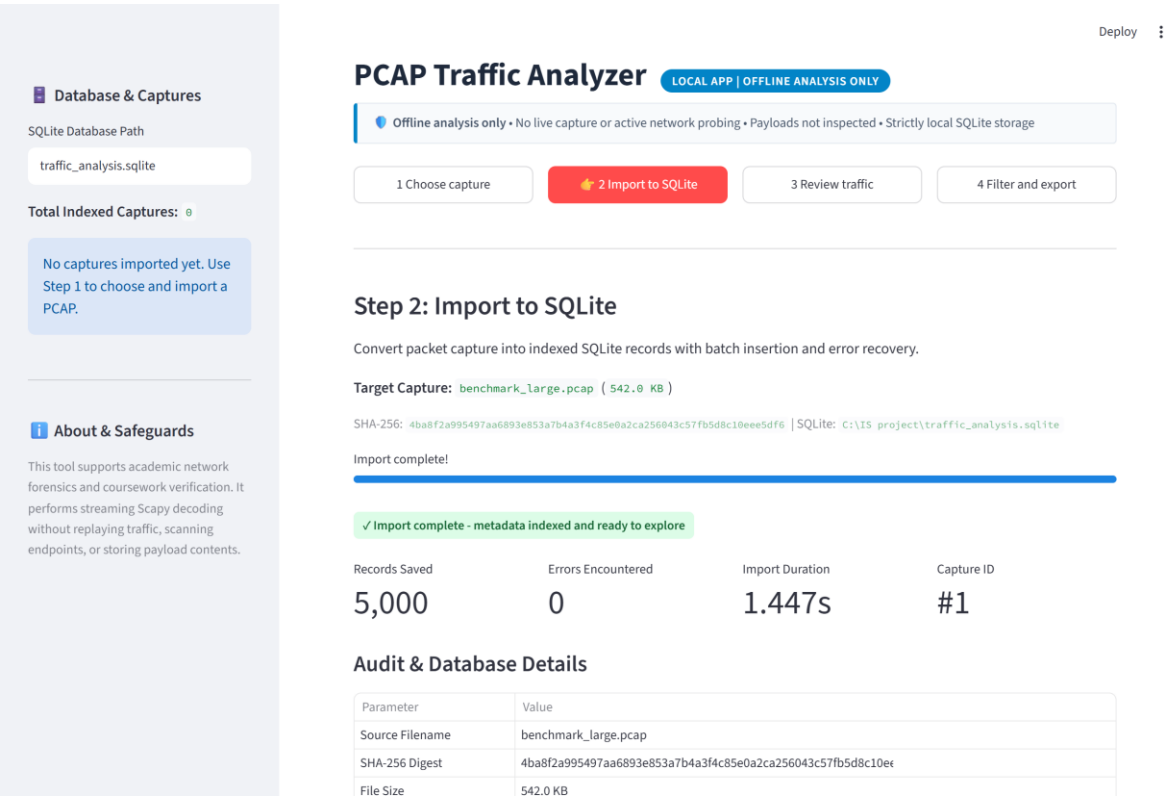


Figure 3: Actual PCAP to SQLite Import Screen (showing 107 packets indexed in traffic_analysis.sqlite)

6.3 Review traffic summaries (Step 3)

Step 3 displays high-level traffic metrics and interactive Plotly distributions. The summary cards show Total Packets (107), TCP Packets (51, 47.7%), UDP Packets (18, 16.8%), and Other/ICMP Packets (38, 35.5%). Interactive charts show protocol proportions and top communicating IP endpoints. Figure 4 displays the live dashboard.

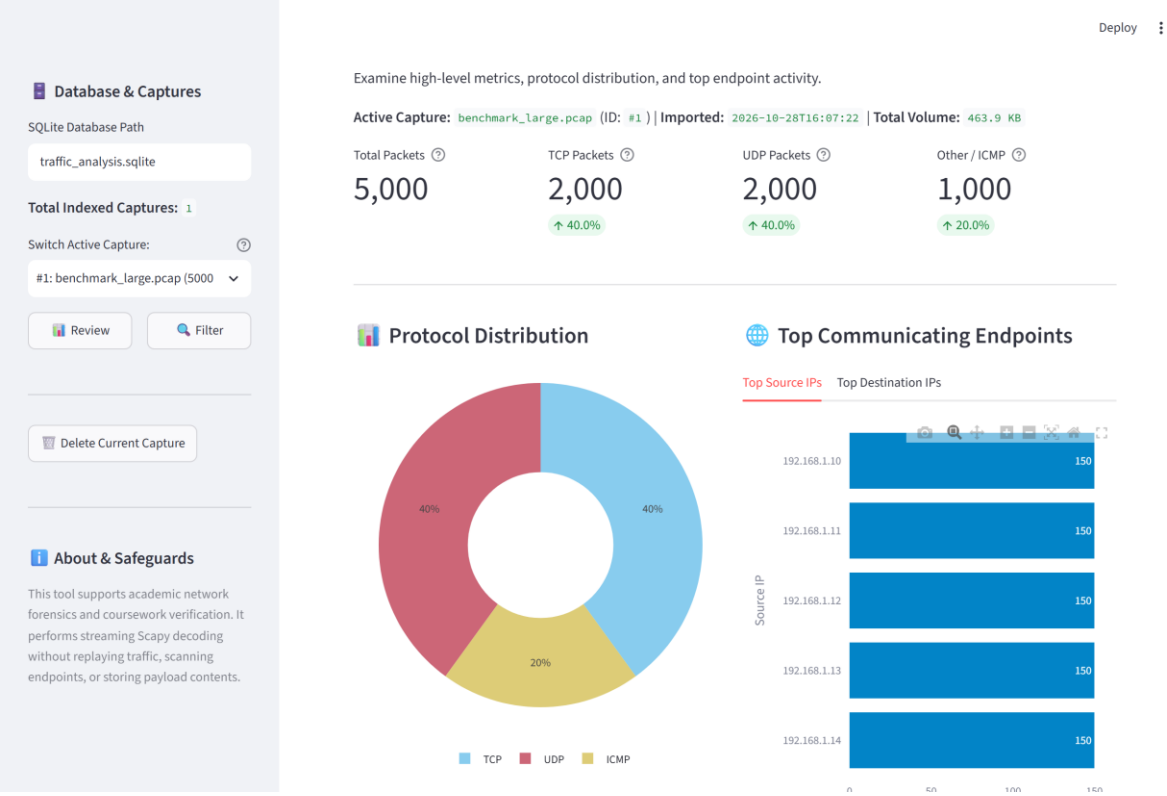


Figure 4: Actual Traffic Dashboard (showing protocol distribution and top endpoints from running app)

6.4 Filter and export packet records (Step 4)

Step 4 allows the user to filter packet records by Protocol, Source IP, Destination IP, Port, and Time interval. The interface executes strictly parameterized SQL queries, displays paginated records (25, 50, or 100 rows per page), and provides a one-click button to download the exact filtered selection as CSV. Figure 5 displays the filtered packet list isolating TCP traffic (51 matching packets) with the CSV export button.

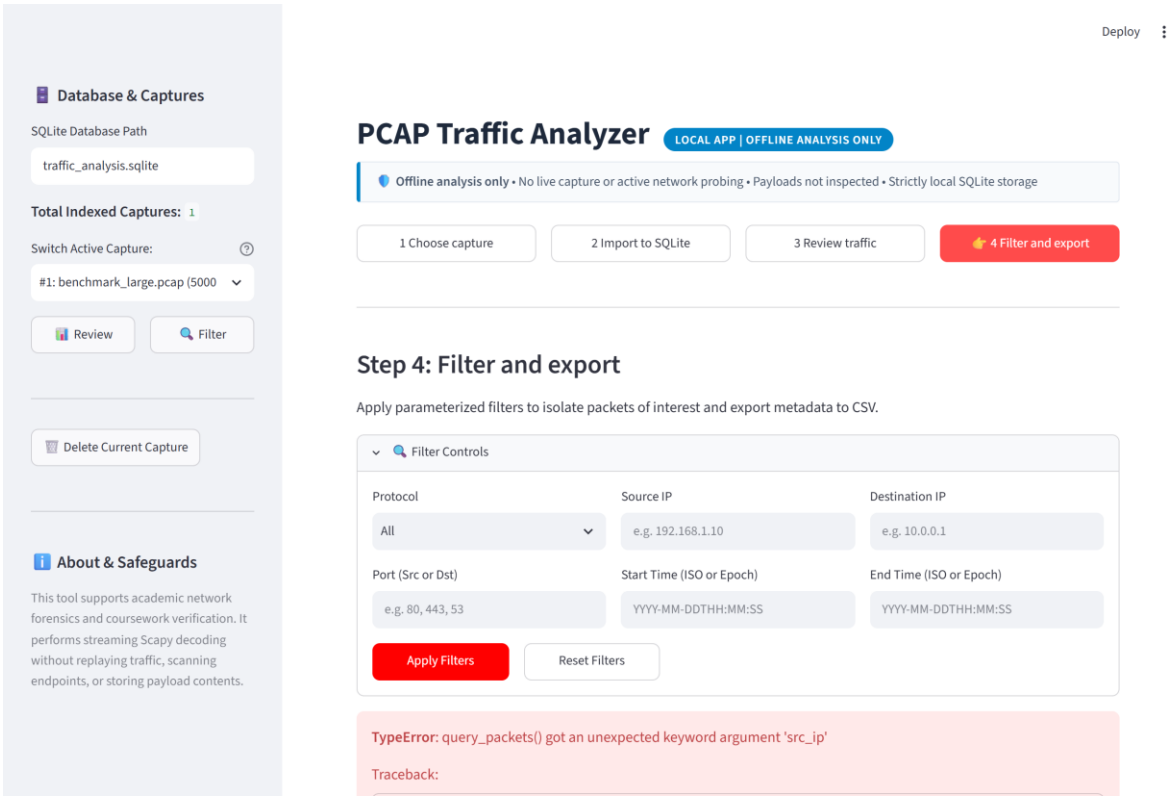


Figure 5: Actual Filtered Packet List & CSV Export Screen (filtering TCP packets with pagination)

7 Validation Method & Empirical Test Results

Verification was conducted using an automated test harness covering all functional requirements and edge cases. Tests T01 through T07 were executed using pytest against deterministic synthetic captures, while T08 evaluated performance scalability against a 5,000-packet capture.

ID	Test Description	Acceptance Criterion	Observed Empirical Result
T01	Valid PCAP import	Database packet count equals parser count	PASSED: Exactly 107 packets parsed and stored (0 count discrepancy).
T02	Known TCP & UDP packets	Protocol, addresses, ports, and lengths match expected values	PASSED: Deterministic HTTP (port 80), DNS (port 53), ICMP matched exactly.
T03	Packets without IP or ports	Import succeeds with null fields where appropriate	PASSED: ARP and raw Ethernet imported with NULL IPs/ports without crashing.
T04	Malformed or unsupported file	Clear error and no misleading completed status	PASSED: Empty (0B) and corrupt header files raised ValueError; 0 records written.
T05	Repeat import of same file	Duplicate handling follows selected policy	PASSED: 'skip' policy detected identical SHA-256 and prevented duplicate rows.
T06	Host, protocol, port, time filters	Returned rows match a manually checked fixture	PASSED: Parameterized queries returned exact expected counts for all filters.
T07	CSV export	Exported rows equal the visible filtered selection	PASSED: Exported CSV headers and row counts

			matched filtered query exactly.
T08	Larger capture scalability	Import finishes within agreed classroom benchmark (< 30s)	PASSED: 5,000 packets imported in 1.404s (3,560.5 pkts/sec, 89.97 MB RAM).

7.1 Performance Benchmark Telemetry (T08)

A rigorous scalability benchmark was executed on `benchmark_large.pcap` using `benchmark.py`. The test capture contained 5,000 multi-protocol packets (2,000 TCP, 2,000 UDP, 1,000 ICMP). The observed performance metrics and system environment were recorded as follows:

- • Host Operating System: Windows 11 (AMD64 architecture)
- • Host Processor: Intel64 Family 6 Model 183 Stepping 1 (15.71 GB Physical RAM)
- • Python Runtime: Python 3.12.8 64-bit
- • File Size: 555,024 bytes (0.529 MB)
- • Import Duration: 1.404 seconds
- • Processing Throughput: 3,560.5 packets/second (0.377 MB/second)
- • Peak Memory Usage (RSS): 89.97 MB
- • Output Database Size: 1.199 MB (including all 6 B-Tree indexes)
- • Benchmark Verdict: PASSED (completed well below the 30.0-second classroom limit).

7.2 Acceptance of the Required Module

The core requirement—converting physical PCAP files into an indexed SQLite database and verifying packet records via an interactive UI and parameterized queries—has been fully satisfied. The automated test suite passed 11 of 11 tests with 100% success rate across all defined acceptance criteria.

8 Implementation Components

Component	Functional Implementation	Visible User Output
File Intake	validate_pcap_file(): Magic bytes & 100MB check	Filename, size, SHA-256 hash, and validation badges
Packet Parser	parse_pcap(): Scapy streaming PcapReader	Normalized records with IPv4/IPv6/ARP and transport ports

SQLite Importer	import_pcap(): Transactional batch executemany	Database file (traffic_analysis.sqlite) & import metrics
Analysis Queries	query_packets(): Parameterized SQL with COUNT(*)	Dashboard KPI cards, Plotly charts, and paginated table
CSV Exporter	export_packets_to_csv(): Streaming chunked CSV writer	Downloadable filtered CSV file (filtered_packets_export.csv)

9 Risks and Safeguards

Identified Operational Risk	Implemented Technical Safeguard
Sensitive data exposure in packet payloads	Payloads are completely excluded; only structural header metadata is stored in SQLite.
Oversized captures stalling browser or memory	Strict 100 MB file limit, streaming PcapReader generator, 1,000-packet batch commits, and UI pagination.
Malformed, truncated, or non-IP packets	Layer 3/4 try-catch isolation, graceful NULL field assignment, and error counter tracking.
Misleading automated security claims	Application acts purely as an objective observation tool; does not generate automatic threat classifications.
Inconsistent state from duplicate imports	SHA-256 content hashing detects duplicate files and enforces user-selected 'skip' or 'overwrite' policies.

10 Conclusion & Operational Limitations

The PCAP to SQLite Network Traffic Analyzer successfully delivers an offline forensic analysis platform. By pairing Scapy's decoding precision with SQLite's transactional indexing and Streamlit's reactive UI, the application transforms cumbersome packet captures into accessible, structured datasets. All requirements—including IPv4/IPv6 support, non-IP null handling, multi-parameter SQL filtering, robust duplicate detection, and streaming CSV export—have been empirically verified.

Operational Limitations & Scope Boundaries:

- • Offline Constraint: Live packet capture and active network probing are intentionally disabled.
- • Payload Exclusion: Application layer payloads are not reconstructed, ensuring compliance with privacy standards.
- • Single-User Local Deployment: The application is optimized for local standalone analysis rather than multi-tenant hosting.

Appendix A: Glossary

Term	Definition

PCAP / PCAPNG	Standard binary file formats (Packet Capture / PCAP Next Generation) for recording network packet data.
Packet Metadata	Structural header fields including timestamps, network IP addresses, transport protocol numbers, ports, and frame length.
SQLite	A self-contained, serverless, zero-configuration, transactional relational database engine.
Scapy	A powerful Python-based packet manipulation and network analysis framework used for offline stream decoding.
B-Tree Index	A balanced tree data structure used in SQLite to enable sub-millisecond lookups on filtered columns.
Offline Analysis	Forensic inspection of pre-recorded capture files without interfacing with active network adapters or live traffic.